

1. findMean(double[] userInputNumbers)

a. Test Case 1: Empty array

- i. Expected output: message saying the array cannot be empty/null and the mean in NaN

ii. Input:

```
1
2 public class StatsTester {
3
4     public static void main(String[] args) {
5         StatsOperations x = new StatsOperations();
6         //set of y values
7         double[] set = {};
8         //set of respective probabilities
9
10        //passing through set of y values
11        System.out.println("Mean: " + x.findMean(set) + "\n\n");
12    }
13 }
```

iii. Output:

```
<terminated> StatsTester [Java Application] C:\Users\s
Invalid input: Array is empty or null.
Mean: NaN
```

b. Test Case 2: Null array

- i. Expected output: message saying the array cannot be empty/null and the mean in NaN

ii. Input:

```
1
2 public class StatsTester {
3
4     public static void main(String[] args) {
5         StatsOperations x = new StatsOperations();
6         //set of y values
7         double[] set = null;
8         //set of respective probabilities
9
10        //passing through set of y values
11        System.out.println("Mean: " + x.findMean(set) + "\n\n");
12    }
13 }
```

iii. Output:

```
Problems @ Javadoc Declaration Console
<terminated> StatsTester [Java Application] C:\Users\s
Invalid input: Array is empty or null.
Mean: NaN
```

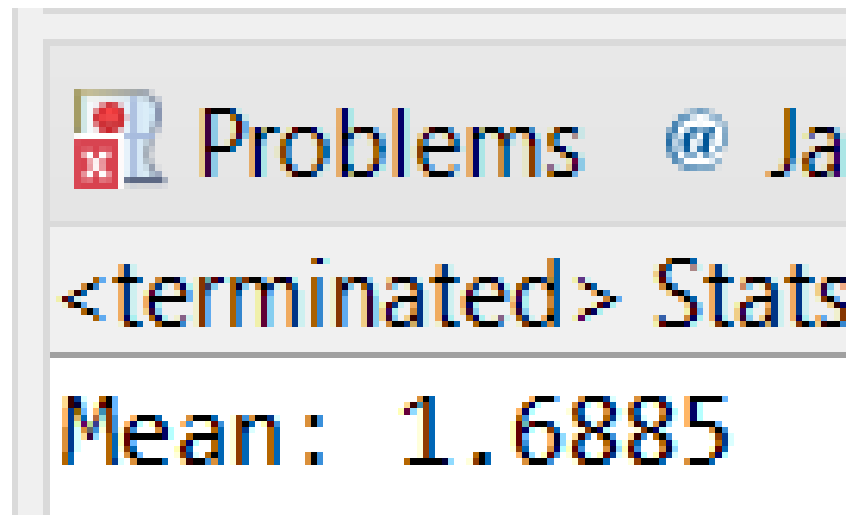
c. Test Case 3: Array with positive and negative decimals: {3.87, 5.22, -8.975, 3.479, 4.8487}

i. Expected Output: $3.87 + 5.22 + -8.975 + 3.479 + 4.8487 = 8.4427$
 $8.4427 / 5 = 1.68854$ (Answer rounded to 4 decimal places)

ii. Input:

```
1 public class StatsTester {
2
3
4 public static void main(String[] args) {
5     StatsOperations x = new StatsOperations();
6     //set of y values
7     double[] set = {3.87, 5.22, -8.975, 3.479, 4.8487};
8     //set of respective probabilities
9
10    //passing through set of y values
11    System.out.println("Mean: " + x.findMean(set) + "\n\n");
12 }
```

iii. Output:



The screenshot shows a Java IDE output window with a header bar that says "Problems @ Ja". Below the header, the text "<terminated> Stats" is displayed. The main output is "Mean: 1.6885".

2. findMedian(double[] userInputNumbers)

a. Test Case 1: Empty array

i. Expected output: message saying the array cannot be empty/null and the median is NaN

ii. Input:

```
1 public class StatsTester {
2
3
4 public static void main(String[] args) {
5     StatsOperations x = new StatsOperations();
6     //set of y values
7     double[] set = {};
8
9     //passing through set of y values
10    System.out.println("Median: " + x.findMedian(set) + "\n\n");
11 }
```

iii. Output:

```
Problems @ Javadoc Declaration Console
<terminated> StatsTester [Java Application] C:\Users\sc
Invalid input: Array is empty or null.
Median: NaN
```

b. Test Case 2: Null array

- i. Expected output: message saying the array cannot be empty/null and the median in NaN

ii. Input:

```
2 public class StatsTester {
3
4 public static void main(String[] args) {
5     StatsOperations x = new StatsOperations();
6     //set of y values
7     double[] set = null;
8
9     //passing through set of y values
10    System.out.println("Median: " + x.findMedian(set) + "\n\n");
11 }
```

iii. Output:

```
Problems @ Javadoc Declaration Console
<terminated> StatsTester [Java Application] C:\Users\sc
Invalid input: Array is empty or null.
Median: NaN
```

c. Test Case 3: Array with odd amount of numbers not in ascending order:

{-68, -7, -37, 8, 42, 199, 86}

i. Expected Output:

Ordered: -68, -37, -7, 8, 42, 86, 199

~~-68, -37, -7, 8, 42, 86, 199~~

8

ii. Input:

```
2 public class StatsTester {
3
4 public static void main(String[] args) {
5     StatsOperations x = new StatsOperations();
6     //set of y values
7     double[] set = {-68, -7, -37, 8, 42, 199, 86};
8
9     //passing through set of y values
10    System.out.println("Median: " + x.findMedian(set) + "\n\n");
11 }
```

iii. Output:

```
Problems @ Ja
<terminated> Stat
Median: 8.0
```

d. Test Case 4: Even amount of numbers not in ascending order:

{74, 92, -137, 1101, 86, -44}

i. Expected Output:

Ordered: -137, -44, 74, 86, 92, 1101

~~-137, -44, 74, 86, 92, 1101~~

$(74 + 86) / 2 = 80$

ii. Input:

```
1
2 public class StatsTester {
3
4 public static void main(String[] args) {
5     StatsOperations x = new StatsOperations();
6     //set of y values
7     double[] set = {74, 92, -137, 1101, 86, -44};
8
9     //passing through set of y values
10    System.out.println("Median: " + x.findMedian(set) + "\n\n");
11}
```

iii. Output:

```
<terminated> StatsTe
Median: 80.0
```

3. findMode(double[] userInputNumbers)

a. Test Case 1: Empty array

i. Expected output: message saying the array cannot be empty/null and the mode is NaN

ii. Input:

```
1
2 public class StatsTester {
3
4 public static void main(String[] args) {
5     StatsOperations x = new StatsOperations();
6     //set of y values
7     double[] set = {};
8
9     //passing through set of y values
10    System.out.println("Mode: " + x.findMode(set) + "\n\n");
11}
```

iii. Output:

```
<terminated> StatsTester [Java Application] C:\Users\pooj...
Invalid input: Array is empty or null.
Mode: null
```

b. Test Case 2: Null array

i. Expected output: message saying the array cannot be empty/null and the mode in NaN

ii. Input:

```
1
2 public class StatsTester {
3
4     public static void main(String[] args) {
5         StatsOperations x = new StatsOperations();
6         //set of y values
7         double[] set = null;
8
9         //passing through set of y values
10        System.out.println("Mode: " + x.findMode(set) + "\n\n");
11    }
```

iii. Output:

```
<terminated> StatsTester [Java Application] C:\Users\pooj...
Invalid input: Array is empty or null.
Mode: null
```

c. Test Case 3: Array with all unique numbers: {1, 2, 3, 4}

i. Expected Output: {1.0, 2.0, 3.0, 4.0}

ii. Input:

```
1
2 public class StatsTester {
3
4     public static void main(String[] args) {
5         StatsOperations x = new StatsOperations();
6         //set of y values
7         double[] set = {1, 2, 3, 4};
8
9         //passing through set of y values
10        System.out.println("Mode: " + x.findMode(set) + "\n\n");
11    }
```

iii. Output:

```
Problems @ Javadoc Declaration Co
<terminated> StatsTester [Java Application] C:\U
Mode: [1.0, 2.0, 3.0, 4.0]
```

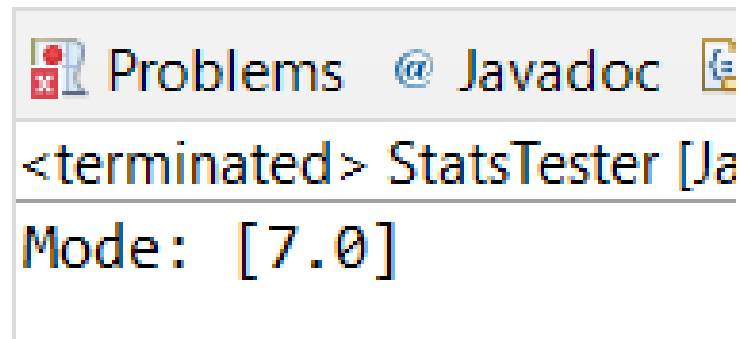
d. Test Case 4: Mode occurs in the last element: {2, 1, 7, 8, 3, 7}

i. Expected Output: {7.0}

ii. Input:

```
1
2 public class StatsTester {
3
4 public static void main(String[] args) {
5     StatsOperations x = new StatsOperations();
6     //set of y values
7     double[] set = {2, 1, 7, 8, 3, 7};
8
9     //passing through set of y values
10    System.out.println("Mode: " + x.findMode(set) + "\n\n");
11}
```

iii. Output:



The screenshot shows an IDE window titled "Problems @ Javadoc". The output text is:
<terminated> StatsTester [Ja
Mode: [7.0]

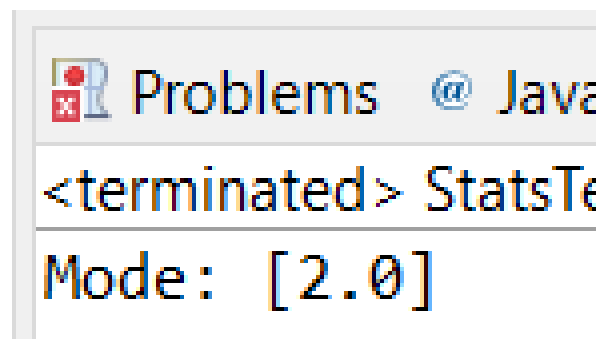
e. Test Case 5: 1 mode: {1, 5, 2, 7, 2, 3, 6}

i. Expected Output: {2.0}

ii. Input:

```
1
2 public class StatsTester {
3
4 public static void main(String[] args) {
5     StatsOperations x = new StatsOperations();
6     //set of y values
7     double[] set = {1, 5, 2, 7, 2, 3, 6};
8
9     //passing through set of y values
10    System.out.println("Mode: " + x.findMode(set) + "\n\n");
11}
```

iii. Output:



The screenshot shows an IDE window titled "Problems @ Java". The output text is:
<terminated> StatsTe
Mode: [2.0]

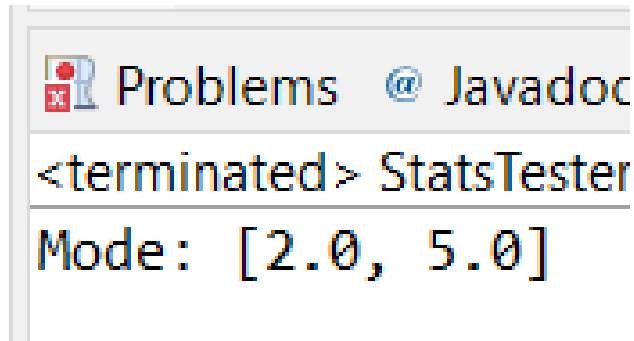
f. Test Case 6: 2 modes: {1, 5, 2, 7, 2, 3, 6, 5}

i. Expected Output: {2.0, 5.0}

ii. Input:

```
1
2 public class StatsTester {
3
4     public static void main(String[] args) {
5         StatsOperations x = new StatsOperations();
6         //set of y values
7         double[] set = {1, 5, 2, 7, 2, 3, 6, 5};
8
9         //passing through set of y values
10        System.out.println("Mode: " + x.findMode(set) + "\n\n");
11    }
```

iii. Output:



```
<terminated> StatsTester
Mode: [2.0, 5.0]
```

4. findSampleVariance(**double**[] userInputNumbers)

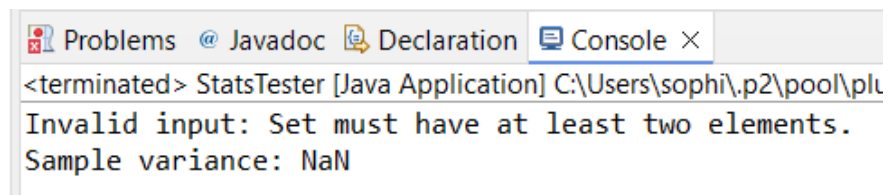
a. Test Case 1: Array with only 1 element

i. Expected output: message saying the array must contains at least 2 elements and the sample variance is NaN

ii. Input:

```
1
2 public class StatsTester {
3
4     public static void main(String[] args) {
5         StatsOperations x = new StatsOperations();
6         //set of y values
7         double[] set = {1};
8
9         //passing through set of y values
10        System.out.println("Sample variance: " + x.findSampleVariance(set) + "\n\n");
11    }
```

iii. Output:



```
<terminated> StatsTester [Java Application] C:\Users\sophi\p2\pool\pl
Invalid input: Set must have at least two elements.
Sample variance: NaN
```

b. Test Case 2: Null array

i. Expected output: message saying the array must contains at least 2 elements and the sample variance is NaN

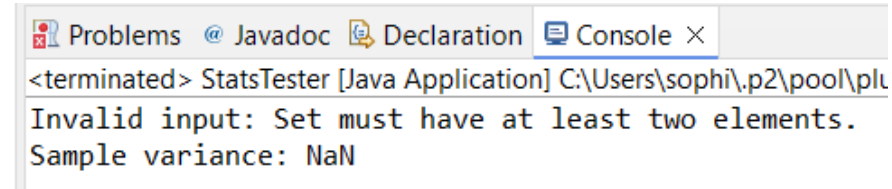
ii. Input:

```

1
2 public class StatsTester {
3
4     public static void main(String[] args) {
5         StatsOperations x = new StatsOperations();
6         //set of y values
7         double[] set = null;
8
9         //passing through set of y values
10        System.out.println("Sample variance: " + x.findSampleVariance(set) + "\n\n");
11

```

iii. Output:



The screenshot shows an IDE window with tabs for Problems, Javadoc, Declaration, and Console. The Console tab is active, displaying the following output:

<terminated> StatsTester [Java Application] C:\Users\sophi\p2\pool\pl

Invalid input: Set must have at least two elements.

Sample variance: NaN

c. Test Case 3: Array with at least 2 elements: {5, -6.2, 3, -7, 2, 4}

i. Expected Output:

Mean: $(5 + -6.2 + 3 + -7 + 2 + 4) / 6 = .1333$

Difference of point and mean: {4.8667, -6.3333, 2.8667, -7.1333, 1.8667, 3.8667}

Squares of differences: {23.6844, 40.1111, 8.2178, 50.8844, 3.4844, 14.9511}

Sum of squares: 141.3333

Sample variance: $141.3333 / 5 = 28.2667$

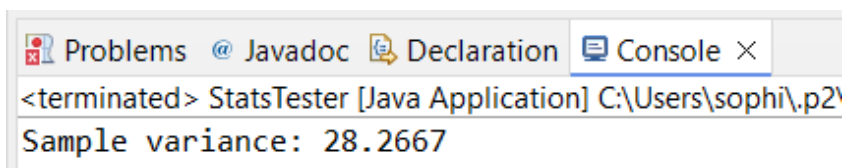
ii. Input:

```

1
2 public class StatsTester {
3
4     public static void main(String[] args) {
5         StatsOperations x = new StatsOperations();
6         //set of y values
7         double[] set = {5, -6.2, 3, -7, 2, 4};
8
9         //passing through set of y values
10        System.out.println("Sample variance: " + x.findSampleVariance(set) + "\n\n");
11

```

iii. Output:



The screenshot shows an IDE window with tabs for Problems, Javadoc, Declaration, and Console. The Console tab is active, displaying the following output:

<terminated> StatsTester [Java Application] C:\Users\sophi\p2\

Sample variance: 28.2667

5. findPopulationVariance(double[] userInputNumbers)

a. Test Case 1: Array with only 1 element

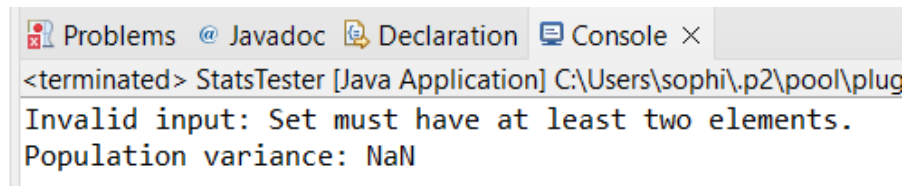
- i. Expected output: message saying the array must contains at least 2 elements and the population variance is NaN
- ii. Input:

```

1
2 public class StatsTester {
3
4     public static void main(String[] args) {
5         StatsOperations x = new StatsOperations();
6         //set of y values
7         double[] set = {1};
8
9         //passing through set of y values
10        System.out.println("Population variance: " + x.findPopulationVariance(set) + "\n\n");
11    }

```

- iii. Output:



```

Problems @ Javadoc Declaration Console ×
<terminated> StatsTester [Java Application] C:\Users\sophi\p2\pool\plug
Invalid input: Set must have at least two elements.
Population variance: NaN

```

b. Test Case 2: Null array

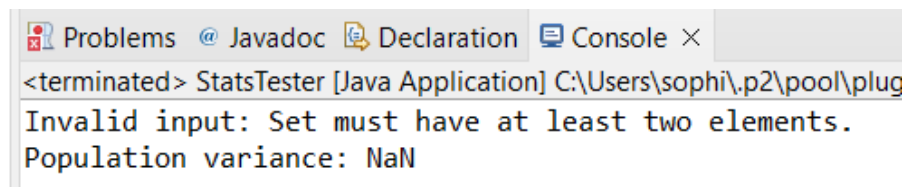
- i. Expected output: message saying the array must contains at least 2 elements and the population variance is NaN
- ii. Input:

```

1
2 public class StatsTester {
3
4     public static void main(String[] args) {
5         StatsOperations x = new StatsOperations();
6         //set of y values
7         double[] set = null;
8
9         //passing through set of y values
10        System.out.println("Population variance: " + x.findPopulationVariance(set) + "\n\n");
11    }

```

- iii. Output:



```

Problems @ Javadoc Declaration Console ×
<terminated> StatsTester [Java Application] C:\Users\sophi\p2\pool\plug
Invalid input: Set must have at least two elements.
Population variance: NaN

```

c. Test Case 3: Array with at least 2 elements: {5, -6.2, 3, -7, 2, 4}

- i. Expected Output:

Mean: $(5 + -6.2 + 3 + -7 + 2 + 4) / 6 = .1333$

Difference of point and mean: {4.8667, -6.3333, 2.8667, -7.1333, 1.8667, 3.8667}

Squares of differences: {23.6844, 40.1111, 8.2178, 50.8844, 3.4844, 14.9511}

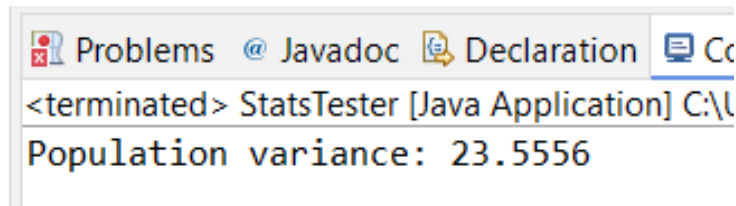
Sum of squares: 141.3333

Sample variance: $141.3333 / 6 = 23.5556$

ii. Input:

```
1
2 public class StatsTester {
3
4     public static void main(String[] args) {
5         StatsOperations x = new StatsOperations();
6         //set of y values
7         double[] set = {5, -6.2, 3, -7, 2, 4};
8
9         //passing through set of y values
10        System.out.println("Population variance: " + x.findPopulationVariance(set) + "\n\n");
11    }
12 }
```

iii. Output:



```
<terminated> StatsTester [Java Application] C:\Users\sophi\p2\pool\plugins\org.eclipse.jdt.launcher\org.eclipse.jdt.launcher.exe
Population variance: 23.5556
```

6. findDiscreteExpectedValue(**double[]** yValues, **double[]** probabilities)

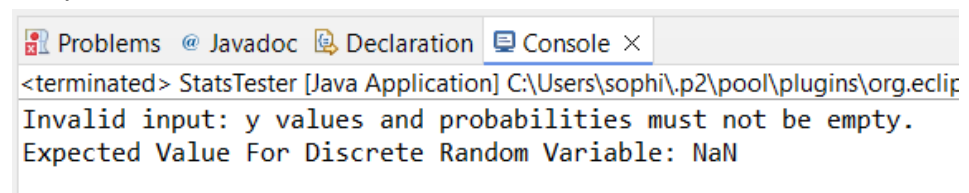
a. Test Case 1: y values array is empty

- i. Expected output: message saying neither arrays can be empty/null and the expected value is NaN

ii. Input:

```
1
2 public class StatsTester {
3
4     public static void main(String[] args) {
5         StatsOperations x = new StatsOperations();
6         //set of y values
7         double[] set = {};
8         //set of respective probabilities
9         double[] p = {.1, .2, .2, .4, .1};
10        //passing through set of y values and respective probabilities
11        System.out.println("Expected Value For Discrete Random Variable: " + x.findDiscreteExpectedValue(set, p) + "\n\n");
12    }
13 }
```

iii. Output:



```
<terminated> StatsTester [Java Application] C:\Users\sophi\p2\pool\plugins\org.eclipse.jdt.launcher\org.eclipse.jdt.launcher.exe
Invalid input: y values and probabilities must not be empty.
Expected Value For Discrete Random Variable: NaN
```

b. Test Case 2: y values array is null

- i. Expected output: message saying neither arrays can be empty/null and the expected value is NaN

ii. Input:

```

1
2 public class StatsTester {
3
4     public static void main(String[] args) {
5         StatsOperations x = new StatsOperations();
6         //set of y values
7         double[] set = null;
8         //set of respective probabilities
9         double[] p = {0.1, 0.2, 0.2, 0.4, 0.1};
10        //passing through set of y values and respective probabilities
11        System.out.println("Expected Value For Discrete Random Variable: " + x.findDiscreteExpectedValue(set, p) + "\n\n");
12

```

iii. Output:

```

Problems @ Javadoc Declaration Console X
<terminated> StatsTester [Java Application] C:\Users\sophi\p2\pool\plugins\org.eclip
Invalid input: y values and probabilities must not be empty.
Expected Value For Discrete Random Variable: NaN

```

c. Test Case 3: probabilities array is empty

i. Expected output: message saying neither arrays can be empty/null and the expected value is NaN

ii. Input:

```

2 public class StatsTester {
3
4     public static void main(String[] args) {
5         StatsOperations x = new StatsOperations();
6         //set of y values
7         double[] set = {1, 2, 3};
8         //set of respective probabilities
9         double[] p = {};
10        //passing through set of y values and respective probabilities
11        System.out.println("Expected Value For Discrete Random Variable: " + x.findDiscreteExpectedValue(set, p) + "\n\n");
12

```

iii. Output:

```

Problems @ Javadoc Declaration Console X
<terminated> StatsTester [Java Application] C:\Users\sophi\p2\pool\plugins\org.eclip
Invalid input: y values and probabilities must not be empty.
Expected Value For Discrete Random Variable: NaN

```

d. Test Case 4: probabilities array is null

i. Expected output: message saying neither arrays can be empty/null and the expected value is NaN

ii. Input:

```

2 public class StatsTester {
3
4     public static void main(String[] args) {
5         StatsOperations x = new StatsOperations();
6         //set of y values
7         double[] set = {1, 2, 3};
8         //set of respective probabilities
9         double[] p = null;
10        //passing through set of y values and respective probabilities
11        System.out.println("Expected Value For Discrete Random Variable: " + x.findDiscreteExpectedValue(set, p) + "\n\n");
12

```

iii. Output:

```

Problems @ Javadoc Declaration Console X
<terminated> StatsTester [Java Application] C:\Users\sophi\p2\pool\plugins\org.eclip
Invalid input: y values and probabilities must not be empty.
Expected Value For Discrete Random Variable: NaN

```

e. Test Case 5: y values array and probability array are different lengths

- i. Expected output: message saying the arrays must be the same length and the expected value is NaN

ii. Input:

```
1 public class StatsTester {
2
3
4 public static void main(String[] args) {
5     StatsOperations x = new StatsOperations();
6     //set of y values
7     double[] set = {1, 2, 3, 4, 5};
8     //set of respective probabilities
9     double[] p = {.25, .25, .25, .25};
10    //passing through set of y values and respective probabilities
11    System.out.println("Expected Value For Discrete Random Variable: " + x.findDiscreteExpectedValue(set, p) + "\n\n");
12 }
```

iii. Output:

```
<terminated> StatsTester [Java Application] C:\Users\sophi\p2\pool\plugins\org.eclipse.justj
Invalid input: Every y value must have a corresponding probability
Expected Value For Discrete Random Variable: NaN
```

f. Test Case 6: There is a negative probability

- i. Expected output: message saying all probabilities must be non-negative and the expected value is NaN

ii. Input:

```
1 public class StatsTester {
2
3
4 public static void main(String[] args) {
5     StatsOperations x = new StatsOperations();
6     //set of y values
7     double[] set = {1, 2, 3, 4};
8     //set of respective probabilities
9     double[] p = {-.25, .75, .25, .25};
10    //passing through set of y values and respective probabilities
11    System.out.println("Expected Value For Discrete Random Variable: " + x.findDiscreteExpectedValue(set, p) + "\n\n");
12 }
```

iii. Output:

```
Problems @ Javadoc Declaration Console X
<terminated> StatsTester [Java Application] C:\Users\sophi\p2\pool\pl
Invalid input: Probabilities must be non-negative.
Expected Value For Discrete Random Variable: NaN
```

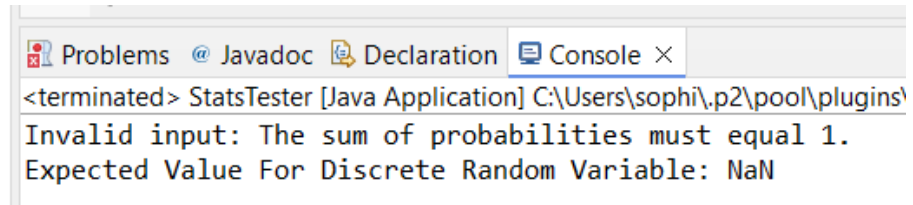
g. Sum of probabilities don't equal 1

- i. Expected output: message saying the probabilities must be equal to 1 and the expected value is NaN

ii. Input:

```
1 public class StatsTester {
2
3
4 public static void main(String[] args) {
5     StatsOperations x = new StatsOperations();
6     //set of y values
7     double[] set = {1, 2, 3, 4};
8     //set of respective probabilities
9     double[] p = {.25, .75, .25, .25};
10    //passing through set of y values and respective probabilities
11    System.out.println("Expected Value For Discrete Random Variable: " + x.findDiscreteExpectedValue(set, p) + "\n\n");
12 }
```

iii. Output:



```
Problems @ Javadoc Declaration Console X
<terminated> StatsTester [Java Application] C:\Users\sophi\p2\pool\plugins\
Invalid input: The sum of probabilities must equal 1.
Expected Value For Discrete Random Variable: NaN
```

h. Valid arrays passed through: {2, 5, 3, 4}, {.3, .1, .4, .2}

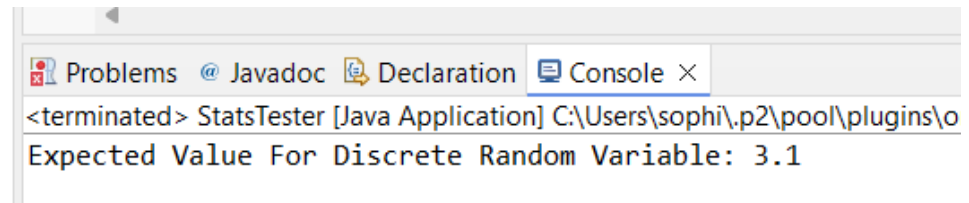
i. Expected output:

$$2(.3) + 5(.1) + 3(.4) + 4(.2) = 3.1$$

ii. Input:

```
1
2 public class StatsTester {
3
4     public static void main(String[] args) {
5         StatsOperations x = new StatsOperations();
6         //set of y values
7         double[] set = {2, 5, 3, 4};
8         //set of respective probabilities
9         double[] p = {.3, .1, .4, .2};
10        //passing through set of y values and respective probabilities
11        System.out.println("Expected Value For Discrete Random Variable: " + x.findDiscreteExpectedValue(set, p) + "\n\n");
12    }
```

iii. Output:



```
Problems @ Javadoc Declaration Console X
<terminated> StatsTester [Java Application] C:\Users\sophi\p2\pool\plugins\o
Expected Value For Discrete Random Variable: 3.1
```

7. findDiscreteVariance(double[] yValues, double[] probabilities)

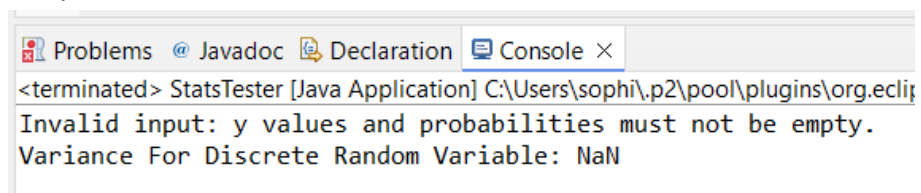
a. Test Case 1: y values array is empty

i. Expected output: message saying neither arrays can be empty/null and the variance is NaN

ii. Input:

```
1
2 public class StatsTester {
3
4     public static void main(String[] args) {
5         StatsOperations x = new StatsOperations();
6         //set of y values
7         double[] set = {};
8         //set of respective probabilities
9         double[] p = {.3, .1, .4, .2};
10        //passing through set of y values and respective probabilities
11        System.out.println("Variance For Discrete Random Variable: " + x.findDiscreteVariance(set, p) + "\n\n");
12    }
```

iii. Output:



```
Problems @ Javadoc Declaration Console X
<terminated> StatsTester [Java Application] C:\Users\sophi\p2\pool\plugins\org.eclij
Invalid input: y values and probabilities must not be empty.
Variance For Discrete Random Variable: NaN
```

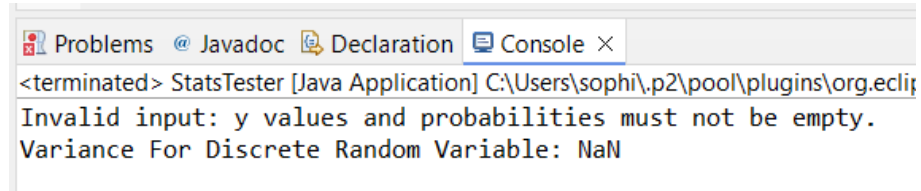
b. Test Case 2: y values array is null

- i. Expected output: message saying neither arrays can be empty/null and the variance is NaN

ii. Input:

```
1
2 public class StatsTester {
3
4@ public static void main(String[] args) {
5     StatsOperations x = new StatsOperations();
6     //set of y values
7     double[] set = null;
8     //set of respective probabilities
9     double[] p = {.3, .1, .4, .2};
10    //passing through set of y values and respective probabilities
11    System.out.println("Variance For Discrete Random Variable: " + x.findDiscreteVariance(set, p) + "\n\n");
12
13 }
```

iii. Output:



```
<terminated> StatsTester [Java Application] C:\Users\sophi\p2\pool\plugins\org.eclipse.jdt.launcher\org.eclipse.jdt.launcher.exe
Invalid input: y values and probabilities must not be empty.
Variance For Discrete Random Variable: NaN
```

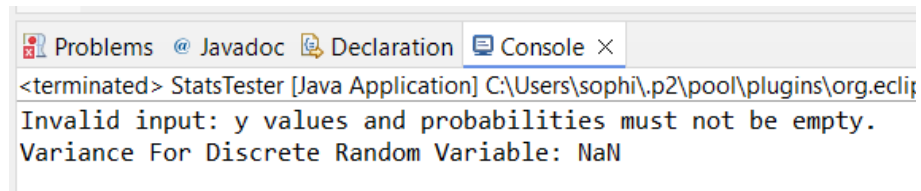
c. Test Case 3: probabilities array is empty

- i. Expected output: message saying neither arrays can be empty/null and the variance is NaN

ii. Input:

```
1
2 public class StatsTester {
3
4@ public static void main(String[] args) {
5     StatsOperations x = new StatsOperations();
6     //set of y values
7     double[] set = {2, 5, 3, 4};
8     //set of respective probabilities
9     double[] p = {};
10    //passing through set of y values and respective probabilities
11    System.out.println("Variance For Discrete Random Variable: " + x.findDiscreteVariance(set, p) + "\n\n");
12
13 }
```

iii. Output:



```
<terminated> StatsTester [Java Application] C:\Users\sophi\p2\pool\plugins\org.eclipse.jdt.launcher\org.eclipse.jdt.launcher.exe
Invalid input: y values and probabilities must not be empty.
Variance For Discrete Random Variable: NaN
```

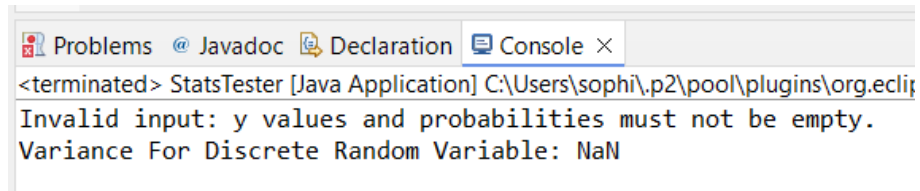
d. Test Case 4: probabilities array is null

- i. Expected output: message saying neither arrays can be empty/null and the variance is NaN

ii. Input:

```
1
2 public class StatsTester {
3
4@ public static void main(String[] args) {
5     StatsOperations x = new StatsOperations();
6     //set of y values
7     double[] set = {2, 5, 3, 4};
8     //set of respective probabilities
9     double[] p = null;
10    //passing through set of y values and respective probabilities
11    System.out.println("Variance For Discrete Random Variable: " + x.findDiscreteVariance(set, p) + "\n\n");
12
13 }
```

iii. Output:

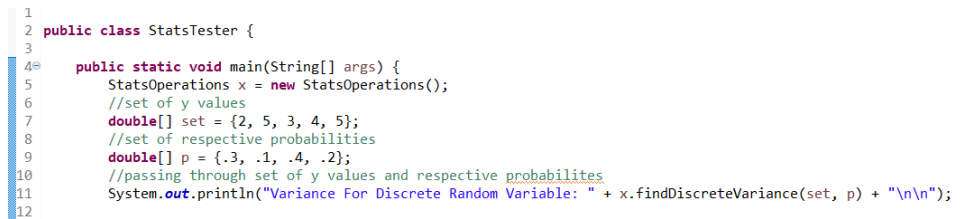


```
<terminated> StatsTester [Java Application] C:\Users\sophi\p2\pool\plugins\org.eclipse.justj.c
Invalid input: y values and probabilities must not be empty.
Variance For Discrete Random Variable: NaN
```

e. Test Case 5: y values array and probability array are different lengths

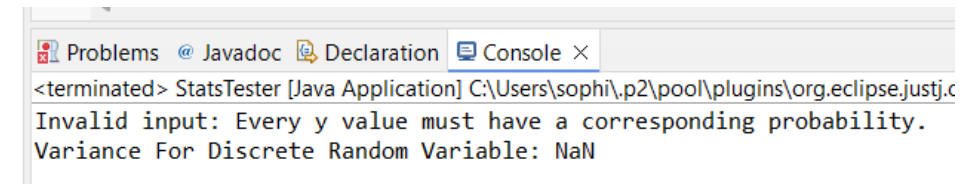
- i. Expected output: message saying the arrays must be the same length and the variance is NaN

ii. Input:



```
1 public class StatsTester {
2
3
4 public static void main(String[] args) {
5 StatsOperations x = new StatsOperations();
6 //set of y values
7 double[] set = {2, 5, 3, 4, 5};
8 //set of respective probabilities
9 double[] p = {.3, .1, .4, .2};
10 //passing through set of y values and respective probabilities
11 System.out.println("Variance For Discrete Random Variable: " + x.findDiscreteVariance(set, p) + "\n\n");
12
```

iii. Output:

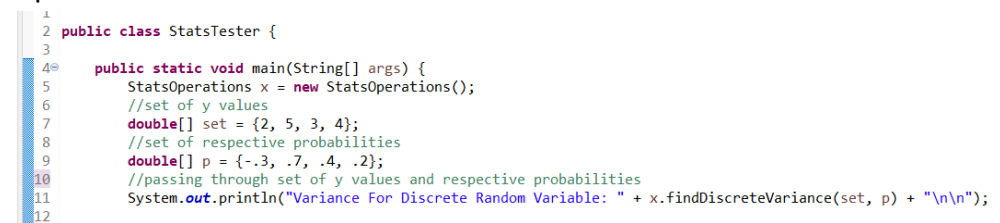


```
<terminated> StatsTester [Java Application] C:\Users\sophi\p2\pool\plugins\org.eclipse.justj.c
Invalid input: Every y value must have a corresponding probability.
Variance For Discrete Random Variable: NaN
```

f. Test Case 6: There is a negative probability

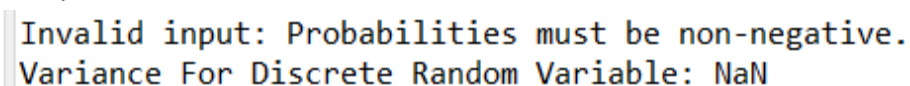
- i. Expected output: message saying all probabilities must be non-negative and the variance is NaN

ii. Input:



```
1 public class StatsTester {
2
3
4 public static void main(String[] args) {
5 StatsOperations x = new StatsOperations();
6 //set of y values
7 double[] set = {2, 5, 3, 4};
8 //set of respective probabilities
9 double[] p = {-.3, .7, .4, .2};
10 //passing through set of y values and respective probabilities
11 System.out.println("Variance For Discrete Random Variable: " + x.findDiscreteVariance(set, p) + "\n\n");
12
```

iii. Output:



```
Invalid input: Probabilities must be non-negative.
Variance For Discrete Random Variable: NaN
```

g. Sum of probabilities don't equal 1

- i. Expected output: message saying the probabilities must be equal to 1 and the variance is NaN

ii. Input:

```

1
2 public class StatsTester {
3
4     public static void main(String[] args) {
5         StatsOperations x = new StatsOperations();
6         //set of y values
7         double[] set = {2, 5, 3, 4};
8         //set of respective probabilities
9         double[] p = {.3, .7, .4, .2};
10        //passing through set of y values and respective probabilities
11        System.out.println("Variance For Discrete Random Variable: " + x.findDiscreteVariance(set, p) + "\n\n");
12    }
13 }

```

iii. Output:

```

Invalid input: The sum of probabilities must equal 1.
Variance For Discrete Random Variable: NaN

```

h. Valid arrays passed through: {2, 5, 3, 4}, {.3, .1, .4, .2}

i. Expected output:

Expected value = $2(.3) + 5(.1) + 3(.4) + 4(.2) = 3.1$

Values squared * probabilities = $4(.3) + 25(.1) + 9(.4) + 16(.2) = 10.5$

Variance = $10.5 - 9.61 = .89$

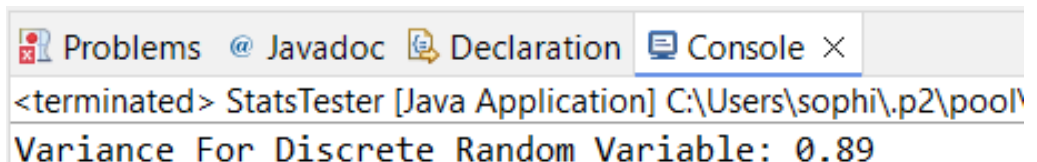
ii. Input:

```

1
2 public class StatsTester {
3
4     public static void main(String[] args) {
5         StatsOperations x = new StatsOperations();
6         //set of y values
7         double[] set = {2, 5, 3, 4};
8         //set of respective probabilities
9         double[] p = {.3, .1, .4, .2};
10        //passing through set of y values and respective probabilities
11        System.out.println("Variance For Discrete Random Variable: " + x.findDiscreteVariance(set, p) + "\n\n");
12    }
13 }

```

iii. Output:



The screenshot shows a Java IDE window with tabs for Problems, Javadoc, Declaration, and Console. The Console tab is active, displaying the output of the StatsTester application. The output is:


```
<terminated> StatsTester [Java Application] C:\Users\sophi\p2\pool\
Variance For Discrete Random Variable: 0.89
```

8. doFactorial(int num)

a. Test Case 1: Negative integer

i. Expected output: message saying the input cannot be non-negative and the factorial is 0

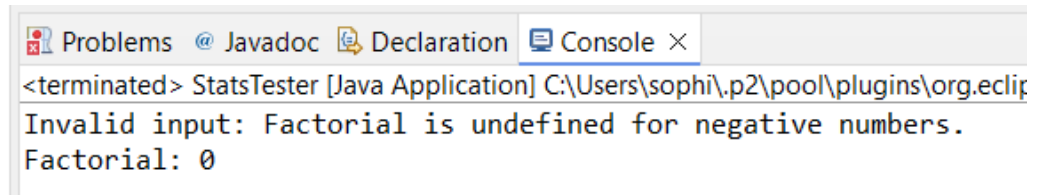
ii. Input:

```

1
2 public class StatsTester {
3
4     public static void main(String[] args) {
5         StatsOperations x = new StatsOperations();
6         //passing through an arbitrary integer
7         System.out.println("Factorial: " + x.doFactorial(-3) + "\n\n");
8     }
9 }

```


iii. Output:

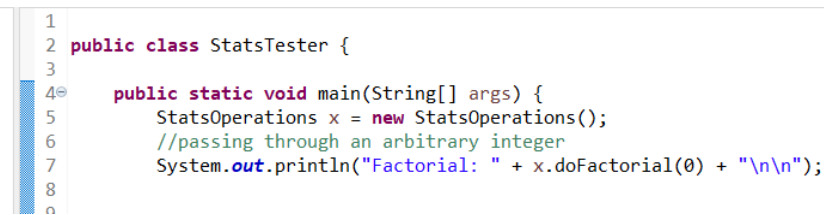


```
<terminated> StatsTester [Java Application] C:\Users\sophi\p2\pool\plugins\org.eclipse.  
Invalid input: Factorial is undefined for negative numbers.  
Factorial: 0
```

b. Test Case 2: 0

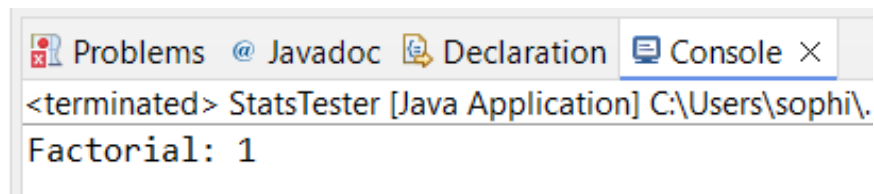
i. Expected output: 1

ii. Input:



```
1 public class StatsTester {  
2  
3  
4 public static void main(String[] args) {  
5 StatsOperations x = new StatsOperations();  
6 //passing through an arbitrary integer  
7 System.out.println("Factorial: " + x.doFactorial(0) + "\n\n");  
8  
9
```

iii. Output:

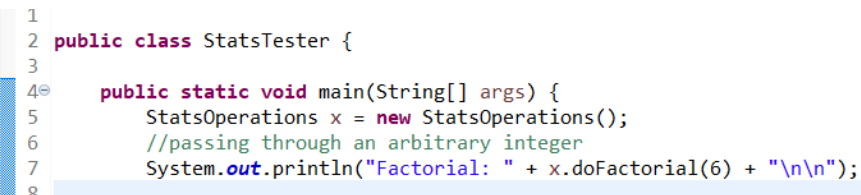


```
<terminated> StatsTester [Java Application] C:\Users\sophi\p2\pool\plugins\org.eclipse.  
Factorial: 1
```

c. Test Case 3: “Typical” number: 6

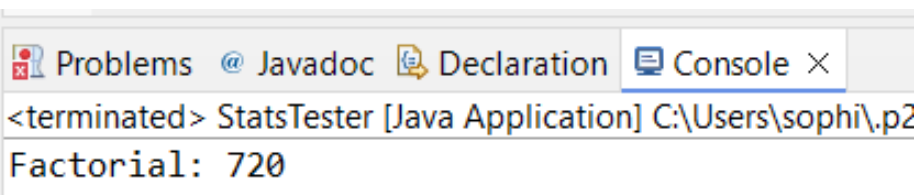
i. Expected output: 720

ii. Input:



```
1 public class StatsTester {  
2  
3  
4 public static void main(String[] args) {  
5 StatsOperations x = new StatsOperations();  
6 //passing through an arbitrary integer  
7 System.out.println("Factorial: " + x.doFactorial(6) + "\n\n");  
8  
9
```

iii. Output:



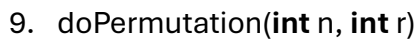
```
<terminated> StatsTester [Java Application] C:\Users\sophi\p2\pool\plugins\org.eclipse.  
Factorial: 720
```

d. Test Case 4: Very large number: 150

i. Expected output: 5.7134×10^{262}

ii. Input:

iii. Output:



i. Expected output: message saying the inputs have to be greater than 0 and n has to be at least r and permutation is 0

```
2 public class StatsTester {
3
4     public static void main(String[] args) {
5         StatsOperations x = new StatsOperations();
6         //passing through arbitrary n and r integers
7         System.out.println("Permutation: " + x.doPermutation(-5, 6) + "\n\n");
8     }
9 }
```

Problems Javadoc Declaration Console ×

<terminated> StatsTester [Java Application] C:\Users\sophi\p2\pool\plugins\c
Invalid input: Ensure $n < 0$, $r < 0$, and n is at least r .
Permutation: 0

i. Expected output: message saying the inputs have to be greater than 0 and n has to be at least r and permutation is 0

```
2 public class StatsTester {
3
4     public static void main(String[] args) {
5         StatsOperations x = new StatsOperations();
6         //passing through arbitrary n and r integers
7         System.out.println("Permutation: " + x.doPermutation(5, -6) + "\n\n");
8     }
9 }
```

iii. Output:

```
Problems @ Javadoc Declaration Console ×
<terminated> StatsTester [Java Application] C:\Users\sophi\p2\pool\plugins\c
Invalid input: Ensure n < 0, r < 0, and n is at least r.
Permutation: 0
```

c. Test Case 3: r is greater than n

i. Expected output: message saying the inputs have to be greater than 0 and n has to be at least r and permutation is 0

ii. Input:

```
2 public class StatsTester {
3
4 public static void main(String[] args) {
5     StatsOperations x = new StatsOperations();
6     //passing through arbitrary n and r integers
7     System.out.println("Permutation: " + x.doPermutation(5, 7) + "\n\n");
8 }
9 }
```

iii. Output:

```
Problems @ Javadoc Declaration Console ×
<terminated> StatsTester [Java Application] C:\Users\sophi\p2\pool\plugins\c
Invalid input: Ensure n < 0, r < 0, and n is at least r.
Permutation: 0
```

d. Test Case 4: All conditions met: n = 15, r = 4

i. Expected output: $P_4^{15} = \frac{15!}{11!} = 15(14)(13)(12) = 32,760$

ii. Input:

```
2 public class StatsTester {
3
4 public static void main(String[] args) {
5     StatsOperations x = new StatsOperations();
6     //passing through arbitrary n and r integers
7     System.out.println("Permutation: " + x.doPermutation(15, 4) + "\n\n");
8 }
9 }
```

iii. Output:

```
Problems @ Javadoc Declaration Console ×
<terminated> StatsTester [Java Application] C:\Users\sophi\p2\
Permutation: 32760
```

10. doCombination(int n, int r)

a. Test Case 1: n is negative

i. Expected output: message saying the inputs have to be greater than 0 and n has to be at least r and combination is 0

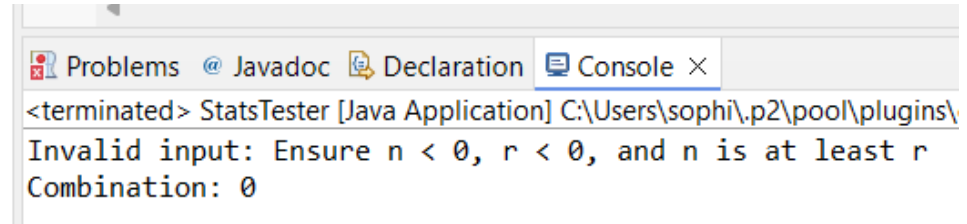
ii. Input:

```

1
2 public class StatsTester {
3
4     public static void main(String[] args) {
5         StatsOperations x = new StatsOperations();
6         //passing through arbitrary n and r integers
7         System.out.println("Combination: " + x.doCombination(-5, 6) + "\n\n");
8

```

iii. Output:



```

<terminated> StatsTester [Java Application] C:\Users\sophi\p2\pool\plugins\
Invalid input: Ensure n < 0, r < 0, and n is at least r
Combination: 0

```

b. Test Case 2: r is negative

- i. Expected output: message saying the inputs have to be greater than 0 and n has to be at least r and combination is 0

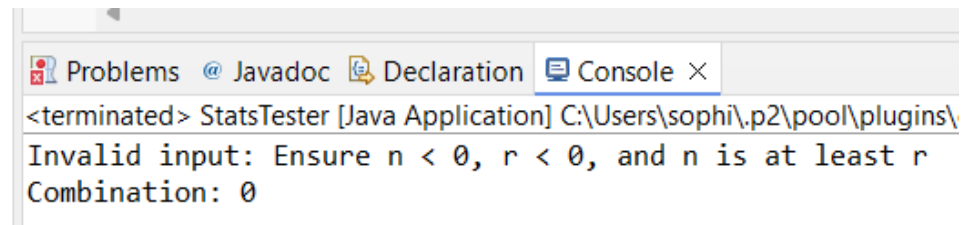
ii. Input:

```

1
2 public class StatsTester {
3
4     public static void main(String[] args) {
5         StatsOperations x = new StatsOperations();
6         //passing through arbitrary n and r integers
7         System.out.println("Combination: " + x.doCombination(5, -6) + "\n\n");
8

```

iii. Output:



```

<terminated> StatsTester [Java Application] C:\Users\sophi\p2\pool\plugins\
Invalid input: Ensure n < 0, r < 0, and n is at least r
Combination: 0

```

c. Test Case 3: r is greater than n

- i. Expected output: message saying the inputs have to be greater than 0 and n has to be at least r and combination is 0

ii. Input:

```

1
2 public class StatsTester {
3
4     public static void main(String[] args) {
5         StatsOperations x = new StatsOperations();
6         //passing through arbitrary n and r integers
7         System.out.println("Combination: " + x.doCombination(5, 7) + "\n\n");
8

```

iii. Output:

```

Problems @ Javadoc Declaration Console X
<terminated> StatsTester [Java Application] C:\Users\sophi\p2\pool\plugins\
Invalid input: Ensure n < 0, r < 0, and n is at least r
Combination: 0

```

d. Test Case 4: All conditions met: $n = 15, r = 4$

i. Expected output: $C_4^{15} = \frac{15!}{4!11!} = 15(14)(13)(.5) = \mathbf{1,365}$

ii. Input:

```

1
2 public class StatsTester {
3
4 public static void main(String[] args) {
5     StatsOperations x = new StatsOperations();
6     //passing through arbitrary n and r integers
7     System.out.println("Combination: " + x.doCombination(15, 4) + "\n\n");
8

```

iii. Output:

```

Problems @ Javadoc Declaration Console X
<terminated> StatsTester [Java Application] C:\Users\sophi\
Combination: 1365

```

11. `getUserChoice(String prompt)`

a. Test Case 1: input is not 1, 2, 3, or 4: 0

i. Expected output: message saying I did not give a valid input of 1, 2, 3, or 4

ii. Input:

```

1
2 public class StatsTester {
3
4 public static void main(String[] args) {
5     StatsOperations x = new StatsOperations();
6     //passing through arbitrary probability double, n int, and r int
7     x.getUserChoice("Choose 1, 2, 3, or 4");
8
9
10
11
12
13
14
15
16
17
18
19
20 //set of y values

```

```

Problems @ Javadoc Declaration Console X
<terminated> StatsTester [Java Application] C:\Users\sophi\p2\pool\plugins\org.eclipse.justj.openjdk.hotsp
Choose 1, 2, 3, or 4
0

```

iii. Output:

```
Invalid input: You must enter a number 1, 2, 3, or 4
```

12. isValidBinomialParameters(double p, int n, int y)

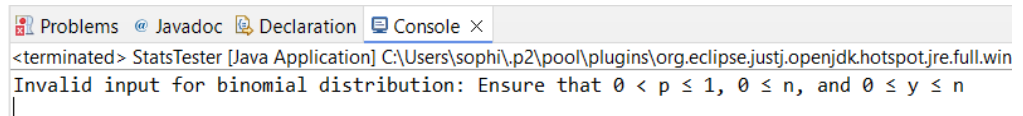
a. Test Case 1: probability of success is 0

- i. Expected output: message saying the probability of success must ball between 0 and 1, only including 1, n and y must be positive, and n must be at least y

ii. Input:

```
1
2 public class StatsTester {
3
4 public static void main(String[] args) {
5     StatsOperations x = new StatsOperations();
6     //passing through arbitrary probability double, n int, and r int
7     x.isValidBinomialParameters(0, 5, 4);
8 }
```

iii. Output:



The screenshot shows the Eclipse IDE's console window. The title bar includes 'Problems', '@ Javadoc', 'Declaration', and 'Console X'. The console output shows the application terminated with the message: 'Invalid input for binomial distribution: Ensure that $0 < p \leq 1$, $0 \leq n$, and $0 \leq y \leq n$ '.

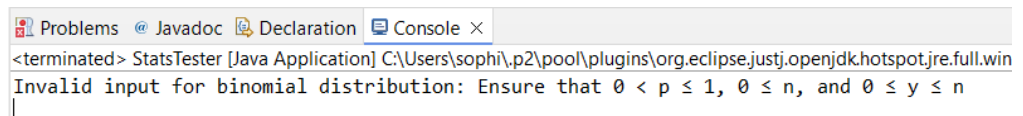
b. Test Case 2: probability of success is greater than 1

- i. Expected output: message saying the probability of success must ball between 0 and 1, only including 1, n and y must be positive, and n must be at least y

ii. Input:

```
1
2 public class StatsTester {
3
4 public static void main(String[] args) {
5     StatsOperations x = new StatsOperations();
6     //passing through arbitrary probability double, n int, and r int
7     x.isValidBinomialParameters(1.2, 5, 4);
8 }
```

iii. Output:



The screenshot shows the Eclipse IDE's console window. The title bar includes 'Problems', '@ Javadoc', 'Declaration', and 'Console X'. The console output shows the application terminated with the message: 'Invalid input for binomial distribution: Ensure that $0 < p \leq 1$, $0 \leq n$, and $0 \leq y \leq n$ '.

c. Test Case 3: n is negative

- i. Expected output: message saying the probability of success must ball between 0 and 1, only including 1, n and y must be positive, and n must be at least y

ii. Input:

```

1
2 public class StatsTester {
3
4     public static void main(String[] args) {
5         StatsOperations x = new StatsOperations();
6         //passing through arbitrary probability double, n int, and r int
7         x.isValidBinomialParameters(.2, -1, 4);
8

```

iii. Output:

```

Problems @ Javadoc Declaration Console x
<terminated> StatsTester [Java Application] C:\Users\sophi\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win
Invalid input for binomial distribution: Ensure that  $0 < p \leq 1$ ,  $0 \leq n$ , and  $0 \leq y \leq n$ 

```

d. Test Case 4: y is negative

- i. Expected output: message saying the probability of success must ball between 0 and 1, only including 1, n and y must be positive, and n must be at least y

ii. Input:

```

1
2 public class StatsTester {
3
4     public static void main(String[] args) {
5         StatsOperations x = new StatsOperations();
6         //passing through arbitrary probability double, n int, and r int
7         x.isValidBinomialParameters(.2, 1, -4);
8

```

iii. Output:

```

Problems @ Javadoc Declaration Console x
<terminated> StatsTester [Java Application] C:\Users\sophi\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win
Invalid input for binomial distribution: Ensure that  $0 < p \leq 1$ ,  $0 \leq n$ , and  $0 \leq y \leq n$ 

```

e. Test Case 5: y is greater than n

- i. Expected output: message saying the probability of success must ball between 0 and 1, only including 1, n and y must be positive, and n must be at least y

ii. Input:

```

3
4     public static void main(String[] args) {
5         StatsOperations x = new StatsOperations();
6         //passing through arbitrary probability double, n int, and r int
7         x.isValidBinomialParameters(.2, 1, 4);
8

```

iii. Output:

```

Problems @ Javadoc Declaration Console x
<terminated> StatsTester [Java Application] C:\Users\sophi\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win
Invalid input for binomial distribution: Ensure that  $0 < p \leq 1$ ,  $0 \leq n$ , and  $0 \leq y \leq n$ 

```

13. isValidGeometricParameters(double p, int y)

a. Test Case 1: probability of success is 0

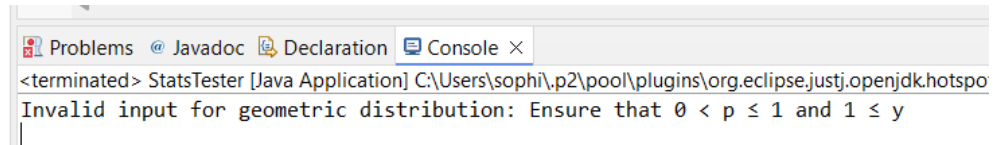
- i. Expected output: message saying the probability of success must ball between 0 and 1, only including 1, and y must be at least 1
- ii. Input:

```

1
2 public class StatsTester {
3
4     public static void main(String[] args) {
5         StatsOperations x = new StatsOperations();
6         //passing through arbitrary probability double, n int, and r int
7         x.isValidGeometricParameters(0, 5);
8     }
9 }

```

- iii. Output:



The screenshot shows the Eclipse IDE's console window. The title bar includes 'Problems', 'Javadoc', 'Declaration', and 'Console'. The console output shows a terminated Java application 'StatsTester' with the following error message: 'Invalid input for geometric distribution: Ensure that $0 < p \leq 1$ and $1 \leq y$ '.

- b. Test Case 2: probability of success is greater than 1

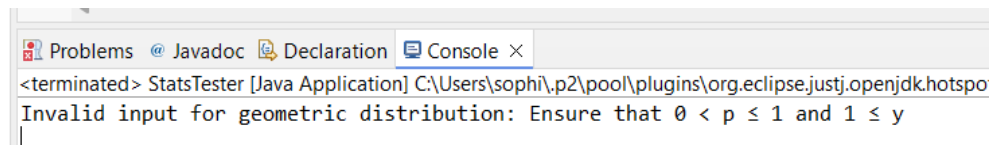
- i. Expected output: message saying the probability of success must ball between 0 and 1, only including 1, and y must be at least 1
- ii. Input:

```

1
2 public class StatsTester {
3
4     public static void main(String[] args) {
5         StatsOperations x = new StatsOperations();
6         //passing through arbitrary probability double, n int, and r int
7         x.isValidGeometricParameters(1.3, 5);
8     }
9 }

```

- iii. Output:



The screenshot shows the Eclipse IDE's console window. The title bar includes 'Problems', 'Javadoc', 'Declaration', and 'Console'. The console output shows a terminated Java application 'StatsTester' with the following error message: 'Invalid input for geometric distribution: Ensure that $0 < p \leq 1$ and $1 \leq y$ '.

- c. Test Case 3: y is less than 1

- i. Expected output: message saying the probability of success must ball between 0 and 1, only including 1, and y must be at least 1
- ii. Input:

```

1
2 public class StatsTester {
3
4     public static void main(String[] args) {
5         StatsOperations x = new StatsOperations();
6         //passing through arbitrary probability double, n int, and r int
7         x.isValidGeometricParameters(.5, 0);
8     }
9 }

```

- iii. Output:


```
Problems @ Javadoc Declaration Console X
<terminated> StatsTester [Java Application] C:\Users\sophi\p2\pool\plugins\org.eclipse.justj.openjdk.hotspo
Invalid input for geometric distribution: Ensure that 0 < p ≤ 1 and 1 ≤ y
```

14. findBinomialDistribution(double p, int n, int y)

** we don't need to test different cases of validity because we already documented such cases for the helper method inside of this method

a. Test Case 1: p = .4, n = 5, y = 3

i. Expected Output:

$$\binom{5}{3} \cdot 4^3 \cdot 6^2 = 10(.064)(.36) = .2304$$

ii. Input:

```
1
2 public class StatsTester {
3
4     public static void main(String[] args) {
5         StatsOperations x = new StatsOperations();
6         //passing through arbitrary probability double, n int, and y int
7         System.out.println("Binomial Distribution: " + x.findBinomialDistribution(.4, 5, 3) + "\n\n");
8     }
```

iii. Output:

```
Problems @ Javadoc Declaration Console X
<terminated> StatsTester [Java Application] C:\Users\sophi\
Binomial Distribution: 0.2304
```

15. findBinomExpectedValue(double p, int n)

** we don't need to test different cases of validity because we already documented such cases for the helper method inside of this method

a. Test Case 1: p = .4, n = 5

i. Expected Output:

$$.4(5) = 2$$

ii. Input:

```
2 public class StatsTester {
3
4     public static void main(String[] args) {
5         StatsOperations x = new StatsOperations();
6         //passing through arbitrary probability double, n int
7         System.out.println("Expected Value for Binomial Distribution: " + x.findBinomExpectedValue(.4, 5) + "\n\n");
8     }
```

iii. Output:

```
Problems @ Javadoc Declaration Console X
<terminated> StatsTester [Java Application] C:\Users\sophi\p2\poc
Expected Value for Binomial Distribution: 2.0
```

16. findBinomVariance(double p, int n)

** we don't need to test different cases of validity because we already documented such cases for the helper method inside of this method

a. Test Case 1: $p = .4, n = 5$

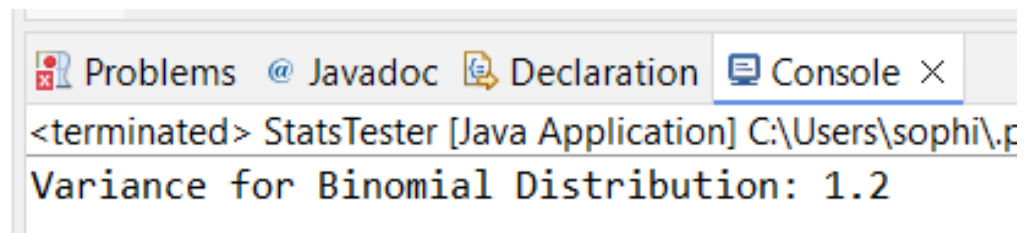
i. Expected Output:

$$.4(5)(.6) = 1.2$$

ii. Input:

```
2 public class StatsTester {
3
4     public static void main(String[] args) {
5         StatsOperations x = new StatsOperations();
6         //passing through arbitrary probability double, n int
7         System.out.println("Variance for Binomial Distribution: " + x.findBinomVariance(.4, 5) + "\n\n");
8     }
9 }
```

iii. Output:



17. findCumulativeBinomialDistribution(double p, int n, int y)

** we don't need to test different cases of validity because we already documented such cases for both of the helper method inside of this method

a. Test Case 1: $p = .4, n = 5$, less than $y = 3$

i. Expected Output:

$$\begin{aligned} \binom{5}{2} \cdot .4^2 \cdot .6^3 &= 10(.16)(.216) = .3456 \\ \binom{5}{1} \cdot .4^1 \cdot .6^4 &= 5(.4)(.1296) = .2592 \\ \binom{5}{0} \cdot .4^0 \cdot .6^5 &= 1(1)(.07776) = .07776 \\ &= \mathbf{.68256} \end{aligned}$$

ii. Input:

```
2 public class StatsTester {
3
4     public static void main(String[] args) {
5         StatsOperations x = new StatsOperations();
6         //passing through arbitrary probability double, n int, int y
7         System.out.println("Cumulative Binomial Distribution: " + x.findCumulativeBinomialDistribution(.4, 5, 3) + "\n\n");
8     }
9 }
```

iii. Output:

```

Problems @ Javadoc Declaration Console X
<terminated> StatsTester [Java Application] C:\Users\sophi\p2\pool\
1. Less than your y value
2. Less than or equal to your y value
3. Greater than your y value
4. Greater than or equal to your y value
(Please type a number)
1
Cumulative Binomial Distribution: 0.6826

```

b. Test Case 2: $p = .4$, $n = 5$, less than or equal to $y = 3$

i. Expected Output:

$$\binom{5}{3} \cdot 4^3 \cdot 6^2 = 10(.064)(.36) = .2304$$

$$\binom{5}{2} \cdot 4^2 \cdot 6^3 = 10(.16)(.216) = .3456$$

$$\binom{5}{1} \cdot 4^1 \cdot 6^4 = 5(.4)(.1296) = .2592$$

$$\binom{5}{0} \cdot 4^0 \cdot 6^5 = 1(1)(.07776) = .07776$$

$$= .91296$$

ii. Input:

```

1
2 public class StatsTester {
3
4     public static void main(String[] args) {
5         StatsOperations x = new StatsOperations();
6         //passing through arbitrary probability double, n int, int y
7         System.out.println("Cumulative Binomial Distribution: " + x.findCumulativeBinomialDistribution(.4, 5, 3) + "\n\n");
8     }
9 }

```

iii. Output:

```

Problems @ Javadoc Declaration Console X
<terminated> StatsTester [Java Application] C:\Users\sophi\p2\p
1. Less than your y value
2. Less than or equal to your y value
3. Greater than your y value
4. Greater than or equal to your y value
(Please type a number)
2
Cumulative Binomial Distribution: 0.913

```

c. Test Case 3: $p = .4$, $n = 5$, greater than $y = 3$

i. Expected Output:

$$\binom{5}{4} \cdot 4^4 \cdot 6^1 = 5(.0256)(.6) = .0768$$

$$\binom{5}{5} \cdot 4^5 \cdot 6^0 = 1(.01024)(1) = .01024$$

$$= .08704$$

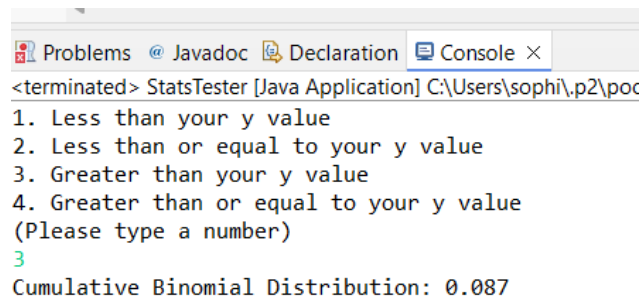
ii. Input:

```

1
2 public class StatsTester {
3
4     public static void main(String[] args) {
5         StatsOperations x = new StatsOperations();
6         //passing through arbitrary probability double, n int, int y
7         System.out.println("Cumulative Binomial Distribution: " + x.findCumulativeBinomialDistribution(.4, 5, 3) + "\n\n");
8     }

```

iii. Output:



```

<terminated> StatsTester [Java Application] C:\Users\sophi\p2\poc
1. Less than your y value
2. Less than or equal to your y value
3. Greater than your y value
4. Greater than or equal to your y value
(Please type a number)
3
Cumulative Binomial Distribution: 0.087

```

d. Test Case 4: p = .4, n = 5, greater than or equal to y = 3

i. Expected Output:

$$\binom{5}{3} \cdot 4^3 \cdot 6^2 = 10(.064)(.36) = .2304$$

$$\binom{5}{4} \cdot 4^4 \cdot 6^1 = 5(.0256)(.6) = .0768$$

$$\binom{5}{5} \cdot 4^5 \cdot 6^0 = 1(.01024)(1) = .01024$$

$$= .31744$$

ii. Input:

```

1
2 public class StatsTester {
3
4     public static void main(String[] args) {
5         StatsOperations x = new StatsOperations();
6         //passing through arbitrary probability double, n int, int y
7         System.out.println("Cumulative Binomial Distribution: " + x.findCumulativeBinomialDistribution(.4, 5, 3) + "\n\n");
8     }
9

```

iii. Output:

```

Problems @ Javadoc Declaration Console X
<terminated> StatsTester [Java Application] C:\Users\sophi\p2\poc
1. Less than your y value
2. Less than or equal to your y value
3. Greater than your y value
4. Greater than or equal to your y value
(Please type a number)
4
Cumulative Binomial Distribution: 0.3174

```

18. findGeometricDistribution(double p, int y)

** we don't need to test different cases of validity because we already documented such cases for the helper method inside of this method

a. Test Case 1: $p = .4$, $y = 4$

i. Expected Output:

$$.6^3 \cdot .4 = .0864$$

ii. Input:

```

1
2 public class StatsTester {
3
4 public static void main(String[] args) {
5     StatsOperations x = new StatsOperations();
6     //passing through arbitrary probability double, int y
7     System.out.println("Geometric Distribution: " + x.findGeometricDistribution(.4, 4) + "\n\n");
8

```

iii. Output:

```

Problems @ Javadoc Declaration Console X
<terminated> StatsTester [Java Application] C:\Users\sophi\p2\
Geometric Distribution: 0.0864

```

19. findGeomExpectedValue(double p, int y)

** we don't need to test different cases of validity because we already documented such cases for the helper method inside of this method

a. Test Case 1: $p = .4$, $y = 4$

i. Expected Output:

$$1/.4 = 2.5$$

ii. Input:

```

1
2 public class StatsTester {
3
4 public static void main(String[] args) {
5     StatsOperations x = new StatsOperations();
6     //passing through arbitrary probability double, int y
7     System.out.println("Expected Value for Geometric Distribution: " + x.findGeomExpectedValue(.4, 4) + "\n\n");
8

```

iii. Output:

```
<terminated> StatsTester [Java Application] C:\Users\sophi\p2\pool\plugins
Expected Value for Geometric Distribution: 2.5
```

20. findGeomVariance(double p, int y)

** we don't need to test different cases of validity because we already documented such cases for the helper method inside of this method

a. Test Case 1: $p = .4, y = 4$

i. Expected Output:

$$.6/.4^2 = 3.75$$

ii. Input:

```
1
2 public class StatsTester {
3
4     public static void main(String[] args) {
5         StatsOperations x = new StatsOperations();
6         //passing through arbitrary probability double, int y
7         System.out.println("Variance for Geometric Distribution: " + x.findGeomVariance(.4, 4) + "\n\n");
8     }
9 }
```

iii. Output:

```
<terminated> StatsTester [Java Application] C:\Users\sophi\p2\pool\pl
Variance for Geometric Distribution: 3.75
```

21. findCumulativeGeometricDistribution(double p, int y)

** we don't need to test different cases of validity because we already documented such cases for both of the helper method inside of this method

a. Test Case 1: $p = .4$, less than $y = 3$

i. Expected Output:

$$.6^1.4 + .6^0.4 = .64$$

ii. Input:

```
1
2 public class StatsTester {
3
4     public static void main(String[] args) {
5         StatsOperations x = new StatsOperations();
6         //passing through arbitrary probability double, int y
7         System.out.println("Cumulative Geometric Distribution: " + x.findCumulativeGeometricDistribution(.4, 3) + "\n\n");
8     }
9 }
```

iii. Output:

```
<terminated> StatsTester [Java Application] C:\Users\sophi\p2\pool\pl
1. Less than your y value
2. Less than or equal to your y value
3. Greater than your y value
4. Greater than or equal to your y value
(Please type a number)
1
Cumulative Geometric Distribution: 0.64
```

b. Test Case 2: $p = .4$, less than or equal to $y = 3$

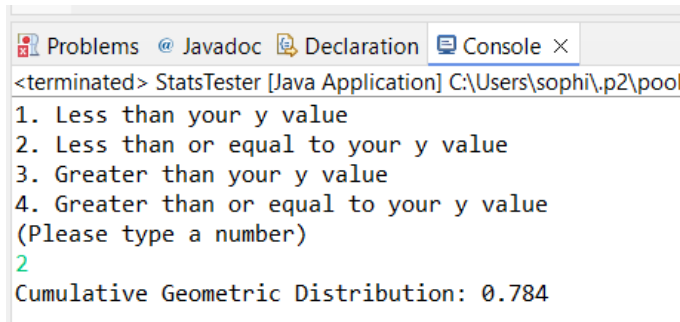
i. Expected Output:

$$.6^2 \cdot .4 + .6^1 \cdot .4 + .6^0 \cdot .4 = .784$$

ii. Input:

```
1
2 public class StatsTester {
3
4     public static void main(String[] args) {
5         StatsOperations x = new StatsOperations();
6         //passing through arbitrary probability double, int y
7         System.out.println("Cumulative Geometric Distribution: " + x.findCumulativeGeometricDistribution(.4, 3) + "\n\n");
8     }
9 }
```

iii. Output:



```
<terminated> StatsTester [Java Application] C:\Users\sophi\p2\pool
1. Less than your y value
2. Less than or equal to your y value
3. Greater than your y value
4. Greater than or equal to your y value
(Please type a number)
2
Cumulative Geometric Distribution: 0.784
```

c. Test Case 3: $p = .4$, greater than $y = 3$

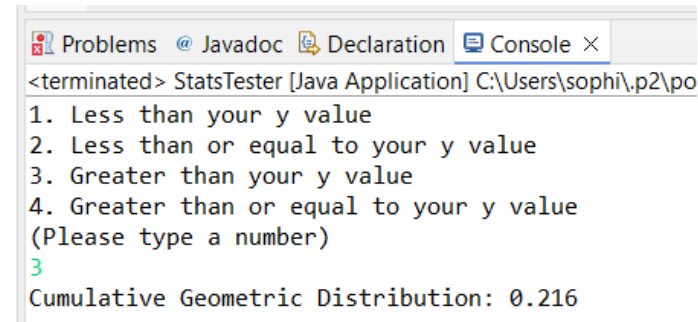
i. Expected Output:

$$1 - .6^2 \cdot .4 + .6^1 \cdot .4 + .6^0 \cdot .4 = .216$$

ii. Input:

```
1
2 public class StatsTester {
3
4     public static void main(String[] args) {
5         StatsOperations x = new StatsOperations();
6         //passing through arbitrary probability double, int y
7         System.out.println("Cumulative Geometric Distribution: " + x.findCumulativeGeometricDistribution(.4, 3) + "\n\n");
8     }
9 }
```

iii. Output:



```
<terminated> StatsTester [Java Application] C:\Users\sophi\p2\po
1. Less than your y value
2. Less than or equal to your y value
3. Greater than your y value
4. Greater than or equal to your y value
(Please type a number)
3
Cumulative Geometric Distribution: 0.216
```

d. Test Case 4: $p = .4$, greater than or equal to $y = 3$

i. Expected Output:

$$1 - .6^1 \cdot 4 + .6^0 \cdot 4 = .36$$

ii. Input:

```
1
2 public class StatsTester {
3
4     public static void main(String[] args) {
5         StatsOperations x = new StatsOperations();
6         //passing through arbitrary probability double, int y
7         System.out.println("Cumulative Geometric Distribution: " + x.findCumulativeGeometricDistribution(.4, 3) + "\n\n");
8     }
9 }
```

iii. Output:

```
Problems @ Javadoc Declaration Console ×
<terminated> StatsTester [Java Application] C:\Users\sophi\p2\pool\
1. Less than your y value
2. Less than or equal to your y value
3. Greater than your y value
4. Greater than or equal to your y value
(Please type a number)
4
Cumulative Geometric Distribution: 0.36
```

22. findHypergeometricDistribution(int N, int n, int r, int y)

a. Test Case 1: N is less than 0

i. Expected Output: A message saying that all inputs must be non-negative, neither n or r can be greater than N, y cannot be greater than either r or n and the hypergeometric distribution will be NaN

ii. Input:

```
1
2 public class StatsTester {
3
4     public static void main(String[] args) {
5         StatsOperations x = new StatsOperations();
6         //passing through arbitrary probability double, int y
7         System.out.println("Hypergeometric Distribution: " + x.findHypergeometricDistribution(-20, 5, 6, 3) + "\n\n");
8     }
9 }
```

iii. Output:

```
Problems @ Javadoc Declaration Console ×
<terminated> StatsTester [Java Application] C:\Users\sophi\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.j
Invalid input: All values must be at least 0, n ≤ N, r ≤ N, y ≤ r, and y ≤ n
Hypergeometric Distribution: NaN
```

b. Test Case 2: n is less than 0

i. Expected Output: A message saying that all inputs must be non-negative, neither n or r can be greater than N, y cannot be greater than either r or n and the hypergeometric distribution will be NaN

ii. Input:


```

1
2 public class StatsTester {
3
4     public static void main(String[] args) {
5         StatsOperations x = new StatsOperations();
6         //passing through arbitrary probability double, int y
7         System.out.println("Hypergeometric Distribution: " + x.findHypergeometricDistribution(20, -5, 6, 3) + "\n\n");
8     }

```

iii. Output:

```

Problems @ Javadoc Declaration Console X
<terminated> StatsTester [Java Application] C:\Users\sophi\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.j
Invalid input: All values must be at least 0,  $n \leq N$ ,  $r \leq N$ ,  $y \leq r$ , and  $y \leq n$ 
Hypergeometric Distribution: NaN

```

c. Test Case 3: r is less than 0

- i. Expected Output: A message saying that all inputs must be non-negative, neither n or r can be greater than N, y cannot be greater than either r or n and the hypergeometric distribution will be NaN

ii. Input:

```

1
2 public class StatsTester {
3
4     public static void main(String[] args) {
5         StatsOperations x = new StatsOperations();
6         //passing through arbitrary probability double, int y
7         System.out.println("Hypergeometric Distribution: " + x.findHypergeometricDistribution(20, 5, -6, 3) + "\n\n");
8     }

```

iii. Output:

```

Problems @ Javadoc Declaration Console X
<terminated> StatsTester [Java Application] C:\Users\sophi\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.j
Invalid input: All values must be at least 0,  $n \leq N$ ,  $r \leq N$ ,  $y \leq r$ , and  $y \leq n$ 
Hypergeometric Distribution: NaN

```

d. Test Case 4: y is less than 0

- i. Expected Output: A message saying that all inputs must be non-negative, neither n or r can be greater than N, y cannot be greater than either r or n and the hypergeometric distribution will be NaN

ii. Input:

```

1
2 public class StatsTester {
3
4     public static void main(String[] args) {
5         StatsOperations x = new StatsOperations();
6         //passing through arbitrary probability double, int y
7         System.out.println("Hypergeometric Distribution: " + x.findHypergeometricDistribution(20, 5, 6, -3) + "\n\n");
8     }

```

iii. Output:

```

Problems @ Javadoc Declaration Console X
<terminated> StatsTester [Java Application] C:\Users\sophi\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.j
Invalid input: All values must be at least 0,  $n \leq N$ ,  $r \leq N$ ,  $y \leq r$ , and  $y \leq n$ 
Hypergeometric Distribution: NaN

```

e. Test Case 5: n is greater than N

- i. Expected Output: A message saying that all inputs must be non-negative, neither n or r can be greater than N, y cannot be greater than either r or n and the hypergeometric distribution will be NaN

ii. Input:

```

1
2 public class StatsTester {
3
4     public static void main(String[] args) {
5         StatsOperations x = new StatsOperations();
6         //passing through arbitrary probability double, int y
7         System.out.println("Hypergeometric Distribution: " + x.findHypergeometricDistribution(20, 21, 6, 3) + "\n\n");
8     }

```

iii. Output:

```

Problems @ Javadoc Declaration Console X
<terminated> StatsTester [Java Application] C:\Users\sophi\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.j
Invalid input: All values must be at least 0,  $n \leq N$ ,  $r \leq N$ ,  $y \leq r$ , and  $y \leq n$ 
Hypergeometric Distribution: NaN

```

f. Test Case 6: r is greater than N

- i. Expected Output: A message saying that all inputs must be non-negative, neither n or r can be greater than N, y cannot be greater than either r or n and the hypergeometric distribution will be NaN

ii. Input:

```

1
2 public class StatsTester {
3
4     public static void main(String[] args) {
5         StatsOperations x = new StatsOperations();
6         //passing through arbitrary probability double, int y
7         System.out.println("Hypergeometric Distribution: " + x.findHypergeometricDistribution(20, 5, 21, 3) + "\n\n");
8     }

```

iii. Output:

```

Problems @ Javadoc Declaration Console X
<terminated> StatsTester [Java Application] C:\Users\sophi\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.j
Invalid input: All values must be at least 0,  $n \leq N$ ,  $r \leq N$ ,  $y \leq r$ , and  $y \leq n$ 
Hypergeometric Distribution: NaN

```

g. Test Case 7: y is greater than r

- i. Expected Output: A message saying that all inputs must be non-negative, neither n or r can be greater than N, y cannot be greater than either r or n and the hypergeometric distribution will be NaN

ii. Input:

```

1
2 public class StatsTester {
3
4     public static void main(String[] args) {
5         StatsOperations x = new StatsOperations();
6         //passing through arbitrary probability double, int y
7         System.out.println("Hypergeometric Distribution: " + x.findHypergeometricDistribution(20, 5, 6, 7) + "\n\n");
8     }

```

iii. Output:

```

Problems @ Javadoc Declaration Console X
<terminated> StatsTester [Java Application] C:\Users\sophi\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.j
Invalid input: All values must be at least 0,  $n \leq N$ ,  $r \leq N$ ,  $y \leq r$ , and  $y \leq n$ 
Hypergeometric Distribution: NaN

```

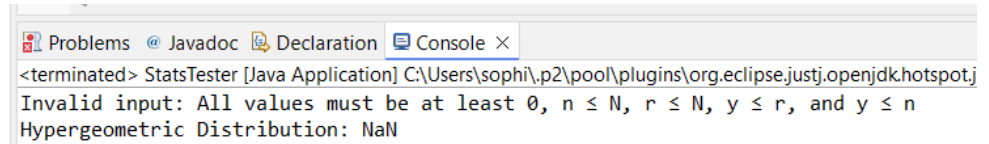
h. Test Case 8: y is greater than n

- i. Expected Output: A message saying that all inputs must be non-negative, neither n or r can be greater than N, y cannot be greater than either r or n and the hypergeometric distribution will be NaN

ii. Input:

```
1 public class StatsTester {
2
3
4 public static void main(String[] args) {
5     StatsOperations x = new StatsOperations();
6     //passing through arbitrary probability double, int y
7     System.out.println("Hypergeometric Distribution: " + x.findHypergeometricDistribution(20, 5, 6) + "\n\n");
8 }
9 }
```

iii. Output:



The screenshot shows the Eclipse IDE console with the following output:
<terminated> StatsTester [Java Application] C:\Users\sophi\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.j
Invalid input: All values must be at least 0, n ≤ N, r ≤ N, y ≤ r, and y ≤ n
Hypergeometric Distribution: NaN

i. Test Case 9: All arguments are valid: N = 20, n = 5, r = 6, y = 3

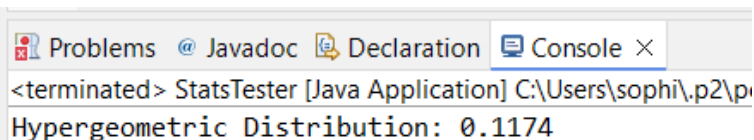
i. Expected Output:

$$\frac{\binom{6}{3}\binom{14}{2}}{\binom{20}{5}} = \frac{20(91)}{15504} = .1174$$

ii. Input:

```
1
2 public class StatsTester {
3
4 public static void main(String[] args) {
5     StatsOperations x = new StatsOperations();
6     //passing through arbitrary probability double, int y
7     System.out.println("Hypergeometric Distribution: " + x.findHypergeometricDistribution(20, 5, 6, 3) + "\n\n");
8 }
9 }
```

iii. Output:



The screenshot shows the Eclipse IDE console with the following output:
<terminated> StatsTester [Java Application] C:\Users\sophi\p2\p
Hypergeometric Distribution: 0.1174

23. findHypergeometricExpectedValue(int N, int n, int r)

** we do not need to check different validity cases for different arguments because the conditional logic is the same as findHypergeometricDistribution(int N, int n, int r, int y)

a. Case 1: N = 20, n = 5, r = 6

i. Expected Output:

$$5(6) / 20 = 1.5$$

ii. Input:

```
1
2 public class StatsTester {
3
4 public static void main(String[] args) {
5     StatsOperations x = new StatsOperations();
6     //passing through arbitrary probability double, int y
7     System.out.println("Expected Value For Hypergeometric Distribution: " + x.findHypergeometricExpectedValue(20, 5, 6) + "\n\n");
8 }
```

iii. Output:

```

Problems @ Javadoc Declaration Console X
<terminated> StatsTester [Java Application] C:\Users\sophi\p2\pool\plugins
Expected Value For Hypergeometric Distribution: 1.5

```

24. findHypergeometricVariance(int N, int n, int r)

** we do not need to check different validity cases for different arguments because the conditional logic is the same as findHypergeometricDistribution(int N, int n, int r, int y)

a. Case 1: N = 20, n = 5, r = 6

i. Expected Output:

$$5 \left(\frac{6}{20} \right) \left(\frac{14}{20} \right) \left(\frac{15}{19} \right) = 5(.3)(.7)(.7895) = .8289$$

ii. Input:

```

1
2 public class StatsTester {
3
4 public static void main(String[] args) {
5     StatsOperations x = new StatsOperations();
6     //passing through arbitrary probability double, int y
7     System.out.println("Variance For Hypergeometric Distribution: " + x.findHypergeometricVariance(20, 5, 6) + "\n\n");
8 }

```

iii. Output:

```

Problems @ Javadoc Declaration Console X
<terminated> StatsTester [Java Application] C:\Users\sophi\p2\pool\plugins\org
Variance For Hypergeometric Distribution: 0.8289

```

25. findNegativeBinomialDistribution(double p, int y, int r)

a. Case 1: probability of success is 0

i. Expected output: a message saying the probability of success must be between 0 and 1, only including 1, y and r have to be at least 1, and y has to be at least r and the negative binomial distribution is NaN

ii. Input:

```

1
2 public class StatsTester {
3
4 public static void main(String[] args) {
5     StatsOperations x = new StatsOperations();
6     //passing through arbitrary probability double, int y
7     System.out.println("Negative Binomial Distribution: " + x.findNegativeBinomialDistribution(0, 5, 3) + "\n\n");
8 }

```

iii. Output:

```

Problems @ Javadoc Declaration Console X
<terminated> StatsTester [Java Application] C:\Users\sophi\p2\pool\plugins\org.eclipse
Invalid input: Ensure that 0 < p ≤ 1, y ≥ 1, r ≥ 1, and y ≥ r
Negative Binomial Distribution: NaN

```

b. Case 2: probability of success is greater than 1

- i. Expected output: a message saying the probability of success must be between 0 and 1, only including 1, y and r have to be at least 1, and y has to be at least r and the negative binomial distribution is NaN

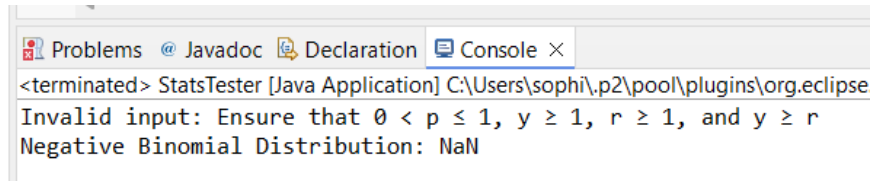
ii. Input:

```

1
2 public class StatsTester {
3
4     public static void main(String[] args) {
5         StatsOperations x = new StatsOperations();
6         //passing through arbitrary probability double, int y
7         System.out.println("Negative Binomial Distribution: " + x.findNegativeBinomialDistribution(1.3, 5, 2) + "\n\n");
8     }
9 }

```

iii. Output:



```

Problems @ Javadoc Declaration Console ×
<terminated> StatsTester [Java Application] C:\Users\sophi\p2\pool\plugins\org.eclipse
Invalid input: Ensure that 0 < p ≤ 1, y ≥ 1, r ≥ 1, and y ≥ r
Negative Binomial Distribution: NaN

```

c. Case 3: y is less than 1

- i. Expected output: a message saying the probability of success must be between 0 and 1, only including 1, y and r have to be at least 1, and y has to be at least r and the negative binomial distribution is NaN

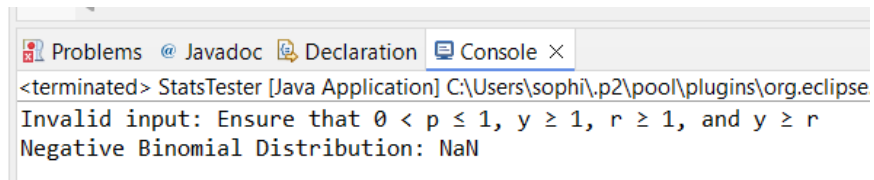
ii. Input:

```

1
2 public class StatsTester {
3
4     public static void main(String[] args) {
5         StatsOperations x = new StatsOperations();
6         //passing through arbitrary probability double, int y
7         System.out.println("Negative Binomial Distribution: " + x.findNegativeBinomialDistribution(.3, 0, 2) + "\n\n");
8     }
9 }

```

iii. Output:



```

Problems @ Javadoc Declaration Console ×
<terminated> StatsTester [Java Application] C:\Users\sophi\p2\pool\plugins\org.eclipse
Invalid input: Ensure that 0 < p ≤ 1, y ≥ 1, r ≥ 1, and y ≥ r
Negative Binomial Distribution: NaN

```

d. Case 4: r is less than 1

- i. Expected output: a message saying the probability of success must be between 0 and 1, only including 1, y and r have to be at least 1, and y has to be at least r and the negative binomial distribution is NaN

ii. Input:

```

1
2 public class StatsTester {
3
4     public static void main(String[] args) {
5         StatsOperations x = new StatsOperations();
6         //passing through arbitrary probability double, int y
7         System.out.println("Negative Binomial Distribution: " + x.findNegativeBinomialDistribution(.3, 5, 0) + "\n\n");
8     }
9 }

```

iii. Output:

```

Problems @ Javadoc Declaration Console X
<terminated> StatsTester [Java Application] C:\Users\sophi\p2\pool\plugins\org.eclipse
Invalid input: Ensure that 0 < p ≤ 1, y ≥ 1, r ≥ 1, and y ≥ r
Negative Binomial Distribution: NaN

```

- e. Case 5: r is greater than y
- Expected output: a message saying the probability of success must be between 0 and 1, only including 1, y and r have to be at least 1, and y has to be at least r and the negative binomial distribution is NaN

ii. Input:

```

1
2 public class StatsTester {
3
4     public static void main(String[] args) {
5         StatsOperations x = new StatsOperations();
6         //passing through arbitrary probability double, int y
7         System.out.println("Negative Binomial Distribution: " + x.findNegativeBinomialDistribution(.3, 5, 6) + "\n\n");
8     }
9 }

```

iii. Output:

```

Problems @ Javadoc Declaration Console X
<terminated> StatsTester [Java Application] C:\Users\sophi\p2\pool\plugins\org.eclipse
Invalid input: Ensure that 0 < p ≤ 1, y ≥ 1, r ≥ 1, and y ≥ r
Negative Binomial Distribution: NaN

```

- f. Case 6: All arguments are valid: p = .3, y = 5, r = 2

i. Expected output:

$$\binom{4}{1} \cdot .3^2 \cdot .7^3 = 4(.09)(.343) = .12348$$

ii. Input:

```

2 public class StatsTester {
3
4     public static void main(String[] args) {
5         StatsOperations x = new StatsOperations();
6         //passing through arbitrary probability double, int y
7         System.out.println("Negative Binomial Distribution: " + x.findNegativeBinomialDistribution(.3, 5, 2) + "\n\n");
8     }
9 }

```

iii. Output:

```

Problems @ Javadoc Declaration Console X
<terminated> StatsTester [Java Application] C:\Users\sophi\p2\pool\plugi
Negative Binomial Distribution: 0.1235

```

26. findNegativeBinomExpectedValue(double p, int r)

** we do not need to check different validity cases for different arguments because the conditional logic is the same as findNegativeBinomialDistribution(double p, int y, int r)

Case 1: p = .3, r = 2

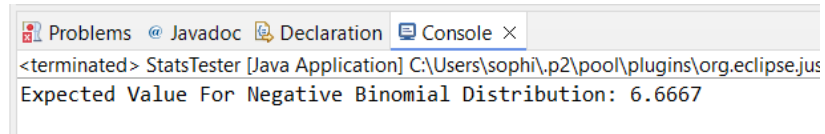
i. Expected Output:

$$2 / .3 = 6.6667$$

ii. Input:

```
1
2 public class StatsTester {
3
4     public static void main(String[] args) {
5         StatsOperations x = new StatsOperations();
6         //passing through arbitrary probability double, int y
7         System.out.println("Expected Value For Negative Binomial Distribution: " + x.findNegativeBinomExpectedValue(.3, 2) + "\n\n");
8     }
9 }
```

iii. Output:



The screenshot shows the Eclipse IDE's Console window. The title bar includes 'Problems', '@ Javadoc', 'Declaration', and 'Console'. The console output displays the result of the Java application execution: '<terminated> StatsTester [Java Application] C:\Users\sophi\p2\pool\plugins\org.eclipse.jus' followed by 'Expected Value For Negative Binomial Distribution: 6.6667'.

27. findNegativeBinomVariance(**double** p, **int** r)

** we do not need to check different validity cases for different arguments because the conditional logic is the same as findNegativeBinomialDistribution(**double** p, **int** y, **int** r)

Case 1: p = .3, r = 2

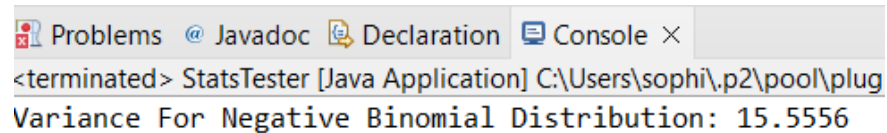
i. Expected Output:

$$2(.7) / .09 = \mathbf{15.5556}$$

ii. Input:

```
1
2 public class StatsTester {
3
4     public static void main(String[] args) {
5         StatsOperations x = new StatsOperations();
6         //passing through arbitrary probability double, int r
7         System.out.println("Variance For Negative Binomial Distribution: " + x.findNegativeBinomVariance(.3, 2) + "\n\n");
8     }
9 }
```

iii. Output:



The screenshot shows the Eclipse IDE's Console window. The title bar includes 'Problems', '@ Javadoc', 'Declaration', and 'Console'. The console output displays the result of the Java application execution: '<terminated> StatsTester [Java Application] C:\Users\sophi\p2\pool\plug' followed by 'Variance For Negative Binomial Distribution: 15.5556'.